

# Automated Attack Knowledge Graph Construction with Large Language Models

Zhihua Wang

Power Dispatching Control Center  
State Grid Shanghai Electric Power  
Company  
Shanghai, China  
wangzh@sh.sgcc.com.cn

Siyuan Fei

Power Dispatching Control Center  
State Grid Shanghai Electric Power  
Company  
Shanghai, China  
503958096@qq.com

Youlin Hu\*

Power Dispatching Control Center  
State Grid Shanghai Electric Power  
Company  
Shanghai, China  
huyoulin@sh.sgcc.com.cn

Dacheng Shan

Nari Group Corporation (State Grid  
Electric Power Research Institute)  
Nanjing NARI Information and  
Communication Technology Co. Ltd.  
Nanjing, Jiangsu, China  
College of Intelligence and  
Computing  
Tianjin University  
Tianjin, Hebei, China  
shandacheng@sgepri.sgcc.com.cn

Shitao Xiao

School of Informatics  
Xiamen University  
Xiamen, Fujian, China  
xiaoshitao@stu.xmu.edu.cn

Lizhao You

School of Informatics  
Xiamen University  
Xiamen, Fujian, China  
lizhaoyou@xmu.edu.cn

Peijun Chen

Nari Group Corporation (State Grid  
Electric Power Research Institute)  
Nanjing NARI Information and  
Communication Technology Co. Ltd.  
Nanjing, Jiangsu, China  
chenpeijun@sgepri.sgcc.com.cn

## Abstract

The construction of attack knowledge graphs aims to transform unstructured Cyber Threat Intelligence (CTI) reports into structured representations, playing a vital role in accurately identifying and swiftly responding to network threats. Automating this process is essential for scalability and efficiency. Existing automated approaches, relying on fixed classifications or ontology-driven methods that require expert tuning, suffer from issues such as limited generalization, high manual overhead, and insufficient semantic understanding. To address these limitations, this paper leverages the superior language comprehension and zero-shot task capabilities of large language models (LLMs) to propose LLM-CAKG, a technology for automated construction of attack knowledge graph based on LLMs. This method uses domain knowledge to guide LLMs to sequentially perform entity extraction and relation recognition in two stages, normalizes and integrates all extracted triples,

and then applies an LLM-driven iterative clustering algorithm to generate the final attack knowledge graph. The accuracy of this method was evaluated using 10 cyber threat reports collected from real-world cases, and the evaluation demonstrates its effectiveness in the automatic construction of attack knowledge graph compared with state-of-the-art approaches.

## CCS Concepts

• **Security and privacy** → Systems security; Vulnerability management.

## Keywords

Unstructured Text Processing, Cyber Attack Knowledge Graph, Automatic Construction Technology, LLMs

\*Corresponding Author.



This work is licensed under a Creative Commons Attribution 4.0 International License. ICCMT 2025, Sanming, China

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1334-7/2025/06

<https://doi.org/10.1145/3757749.3757864>

## ACM Reference Format:

Zhihua Wang, Siyuan Fei, Youlin Hu, Dacheng Shan, Shitao Xiao, Lizhao You, and Peijun Chen. 2025. Automated Attack Knowledge Graph Construction with Large Language Models. In *2025 2nd International Conference on Computer and Multimedia Technology (ICCMT 2025)*, June 06–08, 2025, Sanming, China. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3757749.3757864>

## 1 Introduction

The cybersecurity threat landscape is rapidly escalating. This creates an urgent need for cyber threat intelligence (CTI) analysis and attack pattern identification. Cyber threat reports contain critical information about attack Tactics, Techniques, and Procedures (TTPs). This information is essential for identifying threats and formulating defense strategies. However, valuable intelligence is scattered throughout unstructured texts. Manual extraction is time-consuming and error-prone<sup>[1]</sup>.

Early automated extraction approaches used various methodologies. TTPDrill<sup>[1]</sup> utilized cyber threat ontologies and text similarity scoring, which mapped narrative descriptions to standardized attack techniques. ChainSmith<sup>[2]</sup> extracted Indicators of Compromise (IOCs) and classified them into attack campaign phases. The system linked textual intelligence to multi-stage attack models. The rcATT<sup>[3]</sup> classifier demonstrated systematic classification of natural language text. It worked within the MITRE ATT&CK framework through machine learning approaches.

Recent advances focus on constructing attack knowledge graph from cyber threat reports. EXTRACTOR<sup>[4]</sup> processes cyber incident reports to generate attack behavior graphs. These graphs capture attacker sequences and affected entities. AttackKG<sup>[5]</sup> aggregates attack knowledge from multiple CTI sources into unified technique-centric knowledge graphs. The system automatically extracts entities and relations while merging information across sources.

Despite progress, a significant gap remains between current methods and practical use. Existing approaches rely heavily on predefined rules or extensive data, limiting their adaptability and generalization. Keyword-based TTP classification approaches<sup>[1][2]</sup> may overlook contextual information, while template-driven approaches<sup>[3]</sup> may fail to extract implicit entities. Traditional data-driven approaches such as NLP methods, on the other hand, require extensive labeled datasets for supervised learning. For example, EXTRACTOR<sup>[4]</sup> used 8,000 labeled sentences, while AttackKG<sup>[5]</sup> collected 7,373 examples covering 179 attack techniques. This may cause loss of detecting threat information when facing texts with unknown CTI knowledge. Moreover, traditional data-driven approaches struggle to parse the semantic meaning of complex security texts, offering limited support for coreference resolution and cross-paragraph behavior correlation.

Large Language Models (LLMs) is promising for automated knowledge graph construction. Pretrained on massive data, LLMs have strong contextual understanding and reasoning abilities across domains. They also handle zero-shot and few-shot tasks via instruction following and in-context learning without specialized architectures or specific task training. Therefore, LLMs can effectively address the limitation of generalization, high manual overhead, and insufficient semantic understanding of traditional data-driven approaches. CrystalBall<sup>[6]</sup> has demonstrated the effectiveness of LLM in automated attack graph generation from common vulnerabilities and exposures (CVE) descriptions.

This paper proposes LLM-CAKG, an LLM-based method for automated attack knowledge graph construction from unstructured cyber threat reports. The approach utilizes LLM contextual understanding capabilities guided by cybersecurity domain knowledge to

overcome limitations of previous methods. LLM-CAKG uses LLMs through a two-stage entity-and-relation (triplet) extraction, normalizes extracted triplets, and applies iterative clustering algorithms to construct a comprehensive attack knowledge graph. In summary, the main contributions of this paper are as follows:

- **LLM-guided two-stage extraction method for attack entities and relations:** By incorporating cybersecurity domain knowledge into LLM prompts, LLMs can perform two-stage extraction of attack entities and relations, and accurately identify relevant attack information without requiring task-specific training data.
- **LLM-driven iterative clustering method for aggregating attack entities and relations:** We cluster duplicate entities and relations by automatically identifying and merging semantically equivalent but differently expressed entities and relations using LLMs, enhancing the conciseness of the attack knowledge graph.
- **A complete system LLM-CAKG:** A complete system is designed to standardize and integrate extracted CTI knowledge, transforming scattered attack entities and relations from reports into a unified attack knowledge graph to provide a comprehensive and coherent overview.

We collected real-world cyber threat reports and selected 10 typical CTI ports with varying length, complexity and technical depth for evaluation. We manually analyze and mark entities and relationships to construct the ground-truth attack graph, and use precision, recall rate, and F1 score as metrics. Experimental results show that, compared with the state-of-the-art methods, LLM-CAKG achieves the best precision, recall and F1 scores on the entity extraction and relation extraction, demonstrating the effectiveness of the proposed LLM-CAKG system.

The rest of the paper is organized as follows. Section 2 introduces the background and the related work. Section 3 presents the details of our approach. Section 4 presents the evaluation results. Section 5 concludes this paper.

## 2 BACKGROUND AND RELATED WORK

### 2.1 Cyber Threat Intelligence and Attack Knowledge Graph

**2.1.1 Cyber Threat Intelligence.** Cyber Threat Intelligence (CTI), commonly appearing in technical reports, white papers, blogs, and news articles, serves as a valuable source of information regarding cyber attack. These reports describe numerous aspects of attacks in natural language, including the sequence of operations, impacts on compromised systems, and Indicators of Compromise (IOCs). The knowledge contained within cyber threat intelligence is essential for network security defense<sup>[4]</sup>.

However, numerous cyber threat intelligence reports are written in natural language, lacking unified format and structure (i.e., unstructured). Taking the Cisco Talos report on WeChat browser vulnerabilities<sup>[8]</sup> as an example, critical information (such as attacker names/methods, IOCs, etc.) is scattered across different paragraphs and embedded within natural language descriptions. The CVE identifier "CVE-2023-3420" is merely embedded within a paragraph describing GitHub analysis rather than being separately

listed; the vulnerability trigger mechanism “clicking malicious links to execute JavaScript code” is also dispersed over the middle of paragraphs, requiring contextual inference to identify it as a “one-click exploit.” Moreover, the same concept is referenced using different terminology throughout the report, increasing the difficulty of automated processing.

**2.1.2 Attack Knowledge Graph.** Early approaches represent attack-related information in the form of attack graphs. They guide defenders in implementing defensive measures to protected nodes, and enhance network security<sup>[7]</sup>. Traditional attack graph construction methods require the participation of domain experts. For example, the attack graph generation tool MulVAL<sup>[9]</sup> requires writing Datalog rules manually.

Knowledge graph is a structured semantic knowledge base. It describes concepts in the physical world and their interrelations in symbolic form. The fundamental unit is the “entity-relation-entity” triplet. Entities are interconnected through relations. This forms a network-structured knowledge architecture<sup>[10]</sup>. In the field of cybersecurity, the attack knowledge graph is a graph-structured model used to represent entities and relations related to attacks. It organizes information such as attack steps, attributes, tools, vulnerabilities, and target systems in a structured manner to build a semantic network that supports threat analysis and traceability tracking. In attack path analysis, knowledge graphs can help identify potential attack paths and critical nodes. This enables attack attribution and more effective defense strategies<sup>[11]</sup>. The construction of the attack knowledge graph adopts a structured and top-down approach: it first extracts attack-related entities and relations from cyber threat reports to form structured semantic triples, and then maps these entity and relation semantic triplets to nodes and edges in the attack knowledge graph.

## 2.2 Related Work

Cyber security entity extraction technology has gradually evolved from the stage of predefined rules and regular expressions to the stage based on deep learning methods. Kim et al.<sup>[12]</sup> use a combination of Bidirectional Long Short-Term Memory (BI-LSTM) and Conditional Random Field (CRF) methods. This approach significantly enhanced performance through automatic feature extraction. It also improved entity recognition accuracy in cyber threat intelligence. Gao et al.<sup>[13]</sup> introduce attention mechanisms to improve the accuracy of identifying rare entities. Wang et al.<sup>[14]</sup> combine BERT with knowledge engineering to alleviate issues of entity boundary ambiguity.

Relation extraction is another key step of attack knowledge graph construction. Kim et al.<sup>[15]</sup> propose TIRECO, a joint learning framework that treats coreference relations as predefined relations. It combines cross-sentence relation extraction with a novel pruning strategy. This effectively enhances document-level relation extraction performance. Shen et al.<sup>[16]</sup> construct a supervision model called ResPCNN-ATT that combines deep residual networks with attention mechanisms. It reduces noise interference and enhances the extraction of deep semantic features. Jo et al.<sup>[17]</sup> design the CTI system Vulcan that can extract static threat intelligence data and their semantic relations from unstructured text. As discussed in

Section 1, all these deep learning methods require extensive labelled dataset and extensive supervised learning.

AttacKG+<sup>[18]</sup> is an LLM-based approach for constructing attack knowledge graph and is the most closely related work to ours. It employs an LLM-based rewriting preprocessing to eliminate noise interference. However, as shown in Appendix B of our technical report [19], such rewriting may lose essential information. Moreover, AttacKG+ primarily targets the extraction of tactical-level threat information, while our approach emphasizes the extraction of complete attack processes.

## 3 OUR APPROACH

To address the issues of weak generalization ability, high manual cost, and insufficient semantic understanding in traditional methods for analyzing cyber threat reports and constructing attack knowledge graph, this paper designs LLM-CAKG, an automated attack knowledge graph construction technique based on LLMs. The overall workflow is illustrated in Figure 1. We employ a multi-stage approach driven by LLMs. Specifically, the process begins by collecting unstructured cyber threat reports, and guides the LLM to perform entity extraction and relation identification in stages through domain-specific LLM prompts. Then, we normalize all extracted triples, collect all unique entities and edges, and assemble them into a single graph structure. Finally, LLM-driven iterative clustering algorithm is applied to validate, merge, and assign labels to similar entities, forming the final attack knowledge graph. Below, we introduce these four key modules: entity extraction, relation extraction, aggregation, and clustering. All used LLM prompts are provided in Appendix A of our technical report [19].

### 3.1 Entity Extraction

The first stage focuses on the entity extraction. This is a fundamental step in constructing a cyber attack knowledge graph. This stage utilizes collected unstructured cyber threat report texts as raw input data. These reports come from threat analysis reports publicly released by major threat intelligence platforms or security research institutions. To effectively identify and extract key entity information from these unstructured texts, this stage employs expert knowledge to meticulously design a prompt system. During the entity extraction process, we instruct the LLM to perform deep semantic analysis on the inputted threat report text and require the model to output all detected attack-related entities in a structured format. These entities include the following categories:

- **Malware information:** e.g., malware names, variant identifiers, family classifications, etc.
- **Technical vectors:** e.g., malicious files, scripts, payloads, etc.
- **Attack techniques:** Various exploitation methods, attack tactics, penetration techniques, etc.
- **Vulnerability information:** e.g., CVE numbers, zero-day vulnerability descriptions, affected components, etc.
- **Network indicators:** e.g., IP addresses, domain names, URLs, network communication patterns, etc.
- **Attack subjects and attacked objects:** Groups, organizations, individual identities participating in attacks etc.

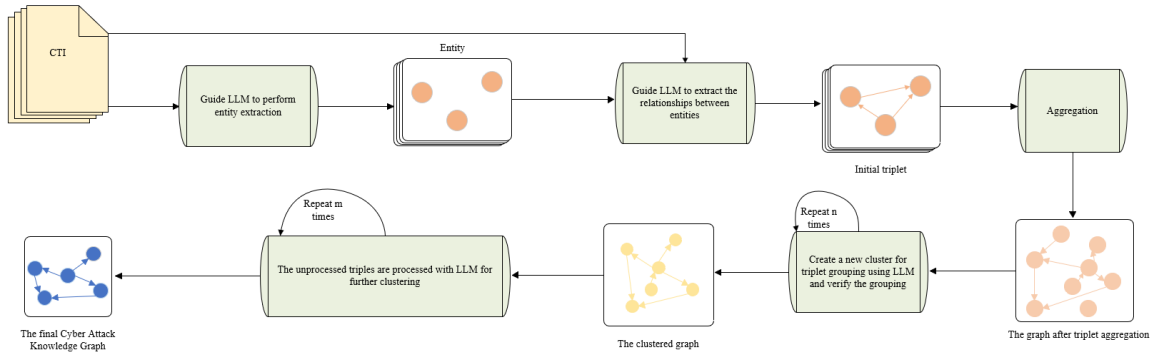


Figure 1: System Workflow of LLM-CAKG.

We impose strict constraints on the model’s output, requiring the model to return only attack-related entity lists while excluding irrelevant text and redundancy. This improves the efficiency and accuracy of information extraction, helping the model focus on key threat intelligence elements and reducing noise.

### 3.2 Relation Extraction

After the entity extraction, LLM-CAKG enters the second stage — relation extraction. This stage is a particularly crucial part in the construction of the network attack knowledge graph and can be regarded as a core component. At this point, we also rely on LLM, but adopt a different prompt strategy. We target to identify various semantic relationships among the entities that have been extracted in the first stage. When extracting relationships, LLM receives two key input contents: one is the entity set extracted from the first stage, and the other is the original threat report text. LLM relies on these two inputs to carry out in-depth context understanding and semantic analysis. In particular, LLM needs to identify many complex relationship types that may exist between entities. These relationship types include the following categories:

- **Causal relations:** an entity causes another entity to happen, or one event starts another event.
- **Temporal relations:** the order and time link between steps in an attack.
- **Conditional relations:** some attack actions happen only when certain conditions are met.
- **Functional associations:** how different entities work together during an attack.
- **Attribution relations:** some tools or systems are used by a certain attack group.

After LLM completes the analysis, the outputs are presented in a standardized “subject-predicate-object” triplet format. This facilitates knowledge graph construction and complies with graph database storage standards. Each triplet represents a basic relational unit in the attack behavior chain. Complete attack chains and attack scenarios can be reconstructed through combinations of these triplets.

Note that we adopt a two-stage processing strategy of entity and relation extraction, rather than performing both simultaneously. The two-stage method reduces the noise interference, and enables

us to efficiently construct preliminary attack behavior knowledge graph from unstructured threat reports. This lays the foundation for subsequent stages.

### 3.3 Aggregation

After completing the two critical stages of entity extraction and relation extraction, aggregation is performed. In the aggregation stage, all triples are cleaned and combined. The same entities, relations from different triples are merged into one single knowledge graph. To reduce redundancy, this process iterates through each input graph, extracting the entities, relations, and edges into three separate sets. By leveraging set’s inherent deduplication feature, it ultimately constructs a new graph object. It should be noted that unlike the previous two stages, the aggregation stage is a purely computational process that does not require LLM participation. Upon completion of aggregation, we obtain a preliminarily integrated attack knowledge graph that contains all extracted entity and relation information from the input report. However, semantically similar entities and relations with different expressions may still exist. These semantic-level redundancies are further optimized in the subsequent stage.

### 3.4 Clustering

After completing the tasks of entity extraction, relation extraction and aggregation, we obtain an original knowledge graph of network attacks. This graph contains a particularly large number of nodes and edges. In threat reports, people often use different terms or expressions to describe the same concept. In the original diagram, there are often entities and relationships that have the same semantics or a high degree of similarity, but are expressed in different ways. This results in a graph structure that is particularly complex and has a high degree of redundancy. Such redundancy can have a negative impact on downstream analysis and application. To address this issue, we have designed an iterative clustering method based on LLM to identify semantically equivalent nodes and edges and merge them. This improves the quality of the graph. During the clustering process, we adopt a progressive iteration strategy rather than expecting the LLM to complete the entire clustering operation at once. We adopt the approach by considering the situation that the terms in the field of network security are very complex and

highly specialized, and LLM has cognitive limitations when dealing with professional concepts. The complete steps of the clustering algorithm based on LLM are as follows:

1) **Initial cluster identification:** given the input of the entity and relation lists from the preliminary attack graph, LLM is guided to group different expressions or synonyms of the same cybersecurity concept into a single cluster.

2) **Cluster validation and labeling:** we use LLM to validate the extracted cluster based on semantic similarity and relevance to known attack techniques. If valid, the cluster is added: its items are removed from the original list, and a shared label is assigned to represent the cluster's meaning.

3) **Iterative clustering:** we repeat steps 1 and 2 until no new clusters are formed after *LOOP\_N* consecutive attempts.

4) **Remaining entities check:** we batch-process remaining entities and relations to see if they fit into existing clusters. The batch size *BATCH\_SIZE* is dynamically adjusted. Criteria include similarity in attack characteristics.

5) **Secondary cluster validation:** we validate newly added entities and relations to ensure that they align with the meaning and scope of existing clusters.

6) **Final graph construction:** we repeat steps 4 and 5 until all items are processed. Then, we generate the final attack knowledge graph.

The clustering algorithm uses a multi-stage validation process instead of relying on fixed similarity thresholds. Guided by domain-specific prompts, the LLM acts as an evaluator to assess semantic equivalence. To avoid over-clustering, we enforce strict criteria that we only merge fully equivalent entities or relations, excluding parent-child or technically distinct terms. Each decision undergoes a secondary check, and uncertain cases are excluded. To avoid under-clustering, we use iterative refinement, batch processing, and global cluster checks to identify overlooked equivalences and ensure proper consolidation of semantically similar entities. Through this LLM-based iterative clustering method, the resulting knowledge graph achieves higher semantic consistency and structural clarity.

## 4 PERFORMANCE EVALUATION

This section evaluates the performance of LLM-CAKG, focusing primarily on the accuracy, completeness, and effectiveness of the framework in extracting attack entities and their interrelations to construct attack knowledge graphs.

### 4.1 Experimental Setup

We collected real-world cyber threat reports from several different sources, including Cisco Talos Intelligence Group<sup>[20]</sup> and Microsoft Security Intelligence Center<sup>[21]</sup>. These reports cover a variety of attack chains differing in type and complexity, ensuring broad representativeness. We selected 10 representative cyber threat reports to evaluate the accuracy of LLM-CAKG in extracting attack knowledge graphs. These reports describe real-world incidents involving complete attack chains and elements such as malware analysis, vulnerability exploitation, and APT activities, varying in length and technical depth. This diversity supports a comprehensive assessment of the framework's adaptability and robustness.

We manually mark entities and relationships and generate the ground-truth attack knowledge graph. We focus on the accuracy and completeness of entity and relationship extraction, using precision, recall rate and F1 score as metrics. During the experiments, the GPT-4o LLM is adopted.

### 4.2 Experimental Results

In real-world attacks, activity characteristics often involve multiple stages, which can be broken down into correlated atomic events. Each atomic event is modeled in the attack knowledge graph as subject-predicate-object triplets. Subjects and objects are attack entities represented as nodes, while predicates are edges that describe semantic relationships between nodes. These triplets are extracted as relations. Based on the manually marked cyber threat reports mentioned above, combining with network security knowledge and the logic of the report. We compare LLM-CAKG with current state-of-the-art open source cyber threat report parsers EXTRACTOR<sup>[4]</sup> and AttackKG<sup>[5]</sup>.

We enable some optimizations in EXTRACTOR for fairness and comprehensiveness of comparison. These optimizations happen when constructing attack graphs based on textual attack descriptions from cyber threat intelligence. The optimizations include activating the co-reference resolution module to improve entity reference consistency. We also allow removal of duplicate nodes to ensure entity and relation extraction quality. We use EXTRACTOR's recommended parameter configurations to ensure optimal performance. AttackKG uses Lowest Common Ancestor (LCA) to calculate distances between entity pairs. Each entity establishes dependency relations with the nearest entities. We adopt a lenient evaluation standard to avoid excessively high false negatives in AttackKG's relation extraction. Any dependency relation between two entities is considered a correct relation.

Table 1 summarizes the statistical results of entity and relation extraction by LLM-CAKG, EXTRACTOR, and AttackKG across 10 cyber threat reports. Columns 2-9 show manually marked baseline, and counts of false negatives and false positives. Rows 13-15 show the calculated overall precision, recall, and F1-scores.

The results show that the recall rate of EXTRACTOR in entity extraction is relatively low, only 0.124. This is because its false negative rate is very high. This situation is mainly due to the design choice of EXTRACTOR, which aggregates all non-IOC entities of the same type into a single entity. Although this design simplifies the graphical representation, it leads to the loss of key information about the attack process. EXTRACTOR uses regular expressions for filtering, which makes it less sensitive to emerging threat entities. It is unable to effectively identify entity variants and new threat metrics, and many key entities are omitted. This aggregation design cannot fully cover the attack scenarios. It inevitably leads to the loss of structural information in the attack graph, which is unfavorable for the utilization of downstream attack knowledge graphs. The overall performance of AttackKG is average and its accuracy is not high. This is mainly due to false alarms when dealing with complex and irrelevant information in cyber threat reports. This has a particularly significant impact on situations that contain a large amount of information unrelated to the attack process. It will

**Table 1: Accuracy of Attack Knowledge Graph Extraction from Cyber Threat Reports.**

| Threat Reports    | Entity Extraction |           |          |              | Relation Extraction |           |          |              |
|-------------------|-------------------|-----------|----------|--------------|---------------------|-----------|----------|--------------|
|                   | Manual            | Extractor | AttackKG | LLM-CAKG     | Manual              | Extractor | AttackKG | LLM-CAKG     |
| APT32             | 7                 | -3(+2)    | -0(+2)   | -0(+0)       | 7                   | -2(+9)    | -1(+0)   | -0(+0)       |
| Frankenstein      | 9                 | -6(+1)    | -1(+2)   | -1(+1)       | 8                   | -3(+6)    | -3(+3)   | -1(+1)       |
| Authorization.exe | 15                | -12(+0)   | -9(+2)   | -0(+0)       | 14                  | -8(+0)    | -9(+2)   | -0(+0)       |
| Attacks On Japan  | 25                | -21(+2)   | -12(+10) | -5(+6)       | 24                  | -13(+7)   | -11(+9)  | -5(+7)       |
| Storm-0940        | 11                | -11(+1)   | -7(+6)   | -0(+0)       | 15                  | -2(+7)    | -10(+4)  | -0(+0)       |
| Foxit PDF Reader  | 16                | -15(+0)   | -7(+8))  | -0(+0)       | 27                  | -23(+9)   | -13(+21) | -0(+0)       |
| Direct Media      | 18                | -18(+0)   | -8(+2)   | -1(+1)       | 15                  | -1(+5)    | -6(+12)  | -3(+6)       |
| MDM               | 10                | -10(+1)   | -5(+17)  | -1(+3)       | 9                   | -2(+18)   | -4(+14)  | -1(+2)       |
| Star Blizzard     | 9                 | -9(+2)    | -3(+7)   | -1(+4)       | 8                   | -7(+15)   | -3(+8)   | -2(+3)       |
| Tencent WeChat    | 17                | -15(+4)   | -9(+11)  | -0(+0)       | 16                  | -15(+5)   | -12(+13) | -1(+1)       |
| Overall Precision | 1.000             | 0.567     | 0.531    | <b>0.895</b> | 1.000               | 0.453     | 0.452    | <b>0.867</b> |
| Overall Recall    | 1.000             | 0.124     | 0.555    | <b>0.934</b> | 1.000               | 0.469     | 0.497    | <b>0.909</b> |
| Overall F1 Score  | 1.000             | 0.204     | 0.543    | <b>0.914</b> | 1.000               | 0.460     | 0.473    | <b>0.887</b> |

Note: In the table, "-X(+Y)" indicates that the number of False Negatives is X and the number of False Positives is Y.

slightly increase the false positive rate, which makes many fragmented subgraphs lack sufficient context to represent the complete attack scenario. Even under relatively lenient evaluation requirements, its relationship extraction performance is still limited, with an f1 score of only 0.473. This reflects the challenges in capturing complex entity relations.

Overall, LLM-CAKG outperforms traditional methods in both entity and relation extraction, achieving near human-level performance. It attains a precision/recall of 0.895/0.934 for entities and 0.867/0.909 for relations, with F1-scores of 0.914 and 0.887 respectively—substantially higher than existing approaches. These results highlight the strengths of LLMs in understanding complex CTI, offering strong generalization and adaptability to novel threat entities and relations.

## 5 CONCLUSION

This paper presents LLM-CAKG, an automated construction method based on LLMs, to convert cyber threat reports to attack knowledge graph. The method leverages carefully designed prompt engineering to harness the natural language understanding capabilities of LLMs, enabling deep parsing of cyber threat intelligence and accurate extraction of attack entity relations. Furthermore, it exploits zero-shot and few-shot learning capabilities through semantic similarity computation and entity relation matching algorithms to achieve intelligent clustering and simplification of attack knowledge. Experimental results demonstrate that LLM-CAKG significantly outperforms traditional methods in information extraction accuracy and knowledge graph completeness. In future, we plan to extract fine-grained information such as attack techniques from the constructed knowledge graph based on LLMs.

## Acknowledgments

This work was supported in part by the Science and Technology Project of State Grid Corporation of China (Grants No. SGSH0000DKJS2400254 and No. 5700-202417382A-3-2-ZX), and

in part by the Fundamental Research Funds for the Central Universities of China (Grant No. 20720240049).

## References

- [1] G. Husari, E. Al-Shaer, M. Ahmed, B. Chu, and X. Niu, "Ttpdrill: Automatic and accurate extraction of threat actions from unstructured text of CTI sources," in Proc. 33rd Annu. Comput. Secur. Appl. Conf., 2017, pp. 103–115.
- [2] Z. Zhu and T. Dumitras, "Chainsmith: Automatically learning the semantics of malicious campaigns by mining threat intelligence reports," in 2018 IEEE Eur. Symp. Secur. Privacy (EuroS&P), 2018, pp. 458–472.
- [3] V. Legoy, M. Caselli, C. Seifert, and A. Peter, "Automated retrieval of ATT&CK tactics and techniques for cyber threat reports," arXiv preprint arXiv:2004.14322, 2020.
- [4] K. Satvat, R. Gjomemo, and V. N. Venkatakrishnan, "Extractor: Extracting attack behavior from threat reports," in 2021 IEEE Eur. Symp. Secur. Privacy (EuroS&P), 2021, pp. 598–615.
- [5] Z. Li, J. Zeng, Y. Chen, Z. Chen, J. Tang, and Z. Wang, "AttackKG: Constructing technique knowledge graph from cyber threat intelligence reports," in Eur. Symp. Res. Comput. Secur., Cham: Springer, 2022, pp. 589–609.
- [6] R. T. Prapty, A. Kundu, and A. Iyengar, "Using retriever augmented large language models for attack graph generation," arXiv preprint arXiv:2408.05855, 2024.
- [7] Z.-w. Ye, Y.-b. Guo, C.-d. Wang, and A.-k. Ju, "Survey on application of attack graph technology," J. Commun., vol. 38, no. 11, pp. 121–132, 2017.
- [8] <https://blog.talosintelligence.com/vulnerability-in-tencent-wechat-custom-browser-could-lead-to-remote-code-execution/>. 2025.
- [9] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A logic-based network security analyzer," in Proc. 14th USENIX Secur. Symp., 2005, pp. 1–16.
- [10] D. Fensel, U. Şimşek, K. Angele, E. Huaman, E. Kärle, G. Panasiuk, I. Toma, J. Umbrich, and A. Wahler, "Introduction: What is a knowledge graph?" in Knowledge Graphs: Methodology, Tools and Selected Use Cases, Eds. Cham: Springer, 2020, pp. 1–10.
- [11] L. F. Sikos, "Cybersecurity knowledge graphs," Knowl. Inf. Syst., vol. 65, no. 9, pp. 3511–3531, 2023.
- [12] G. Kim, C. Lee, J. Jo, and H. Lim, "Automatic extraction of named entities of cyber threats using a deep Bi-LSTM-CRF network," Int. J. Mach. Learn. Cybern., vol. 11, pp. 2341–2355, 2020.
- [13] C. Gao, X. Zhang, and H. Liu, "Data and knowledge-driven named entity recognition for cyber security," Cybersecurity, vol. 4, no. 1, p. 9, 2021.
- [14] X. Wang, R. Liu, J. Yang, K. Liu, and Y. Sun, "Cyber threat intelligence entity extraction based on deep learning and field knowledge engineering," in Proc. 2022 IEEE 25th Int. Conf. Comput. Supported Coop. Work Design (CSCWD), 2022, pp. 406–413.
- [15] X. Wang, M. Xiong, Y. Luo, L. Wang, and F. Dai, "Joint learning for document-level threat intelligence relation extraction and coreference resolution based on GCN," in Proc. 2020 IEEE 19th Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom), 2020, pp. 584–591.

- [16] G. Shen, W. Wang, Q. Mu, Y. Hou, and X. Guan, "Data-driven cybersecurity knowledge graph construction for industrial control system security," *Wireless Commun. Mobile Comput.*, vol. 2020, Art. no. 8883696, 2020.
- [17] H. Jo, Y. Lee, and S. Shin, "Vulcan: Automatic extraction and analysis of cyber threat intelligence from unstructured text," *Comput. Secur.*, vol. 120, p. 102763, 2022.
- [18] Y. Zhang, T. Du, Y. Ma, J. Yan, S. Li, and Z. Li, "AttacKG+: Boosting attack knowledge graph construction with large language models," *arXiv preprint arXiv: 2405.04753*, 2024.
- [19] <https://github.com/youlizhao/llm-akg/blob/main/appendix.pdf>. 2025.
- [20] <https://blog.talosintelligence.com>. 2025.
- [21] <https://www.microsoft.com/en-us/security/blog/topic/threat-intelligence>. 2025.